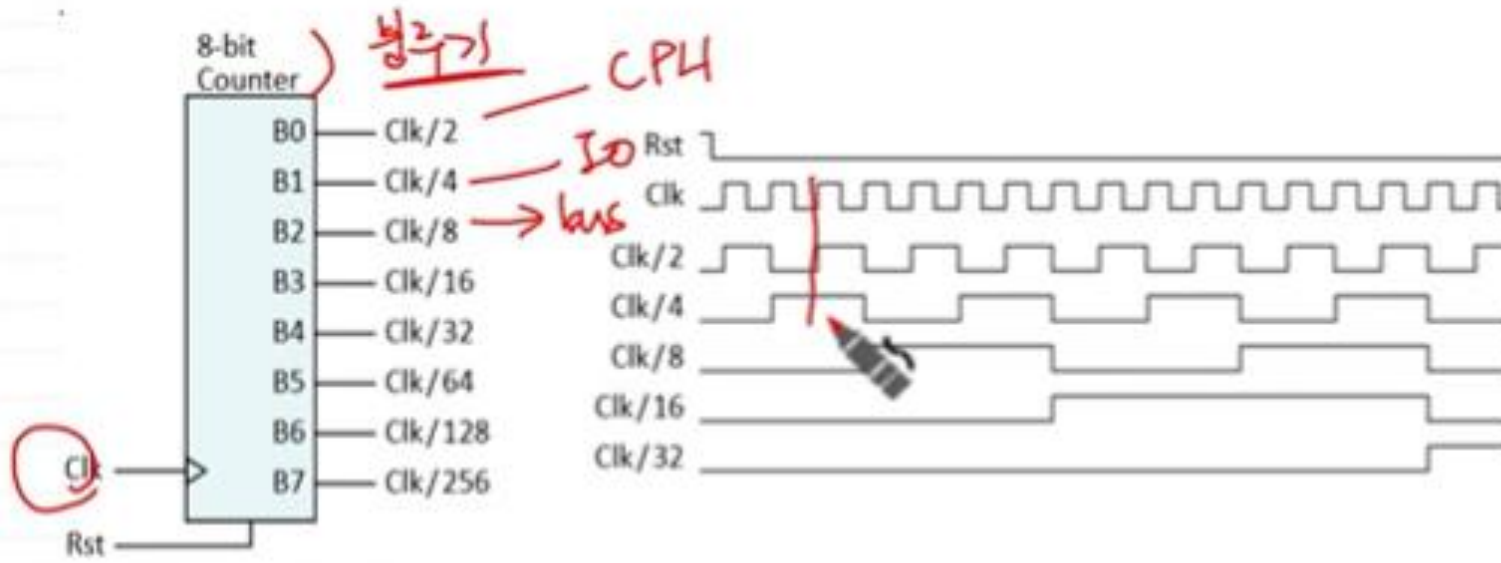


10/07일 프로젝트

참조 파일 – ~/1007_source_file
~/constraint/FND_COU.xdc

개념 이해

clk distributer -> 이미 IP별로 있다



질문에 대한 공부

1. clk distributer는 각 clk끼리 어떻게 타이밍을 맞추나?

Clock skew를 어떻게 잡아내나?

내부 ip 로직이 어떻게 되어있는가

개념 이해

Tcl ConsoleMessagesLogReportsDesign RunsPowerMethodologyTiming ×

Q

⌵

⌶

●

Design Timing Summary

General Information

Timer Settings

Design Timing Summary

Clock Summary (1)

> Check Timing (29)

> Intra-Clock Paths

Inter-Clock Paths

Other Path Groups

User Ignored Paths

Unconstrained Paths

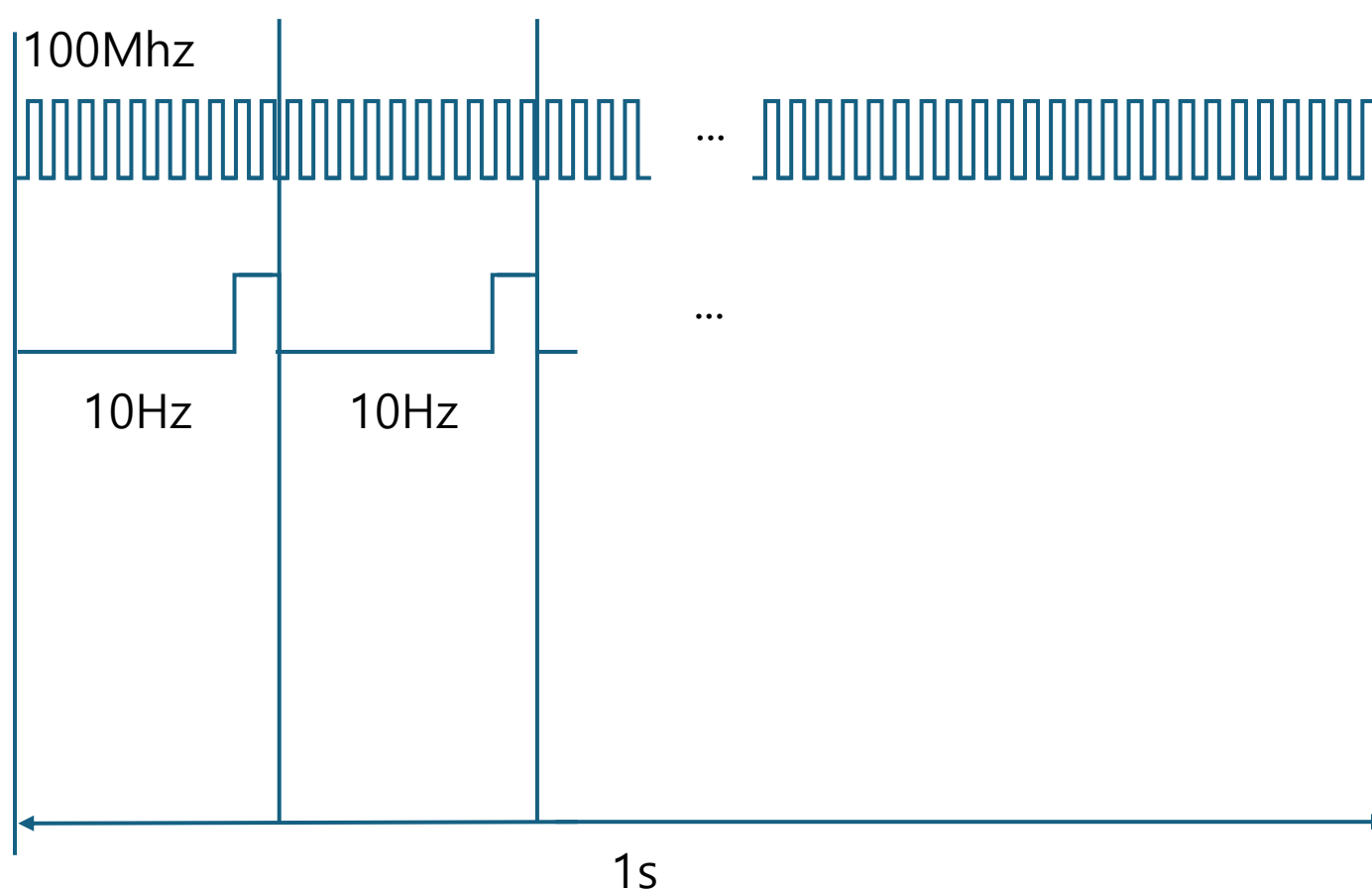
Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 4.652 ns	Worst Hold Slack (WHS): 0.247 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 71	Total Number of Endpoints: 71	Total Number of Endpoints: 58

All user specified timing constraints are met.

2. implementation에 나오는 이 옵션들이 가지는 의미는 무엇

개념 이해

3. 현업에서 tick 을 왜 쓰는가 ?



Clock을 분주해서 카운터로 구성할 때
어떤 문제점이 있는지 설명

Tick 의 장점

* 프로젝트 tic generation

100MHz를 Fully하게 쓰고 싶다 (Tick으로 신호를 전달한다.)
일의 단위를 만드는 것(신호를 제어할 때)

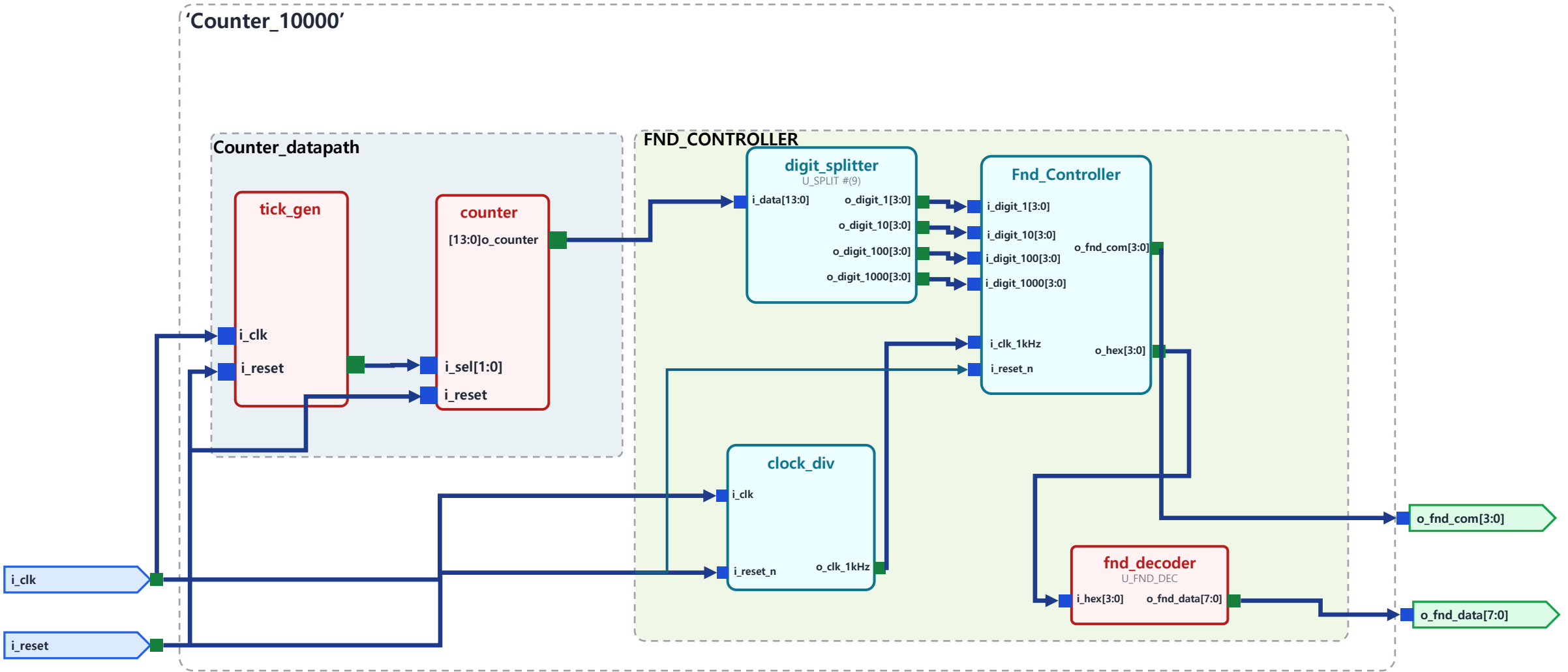
개념 이해

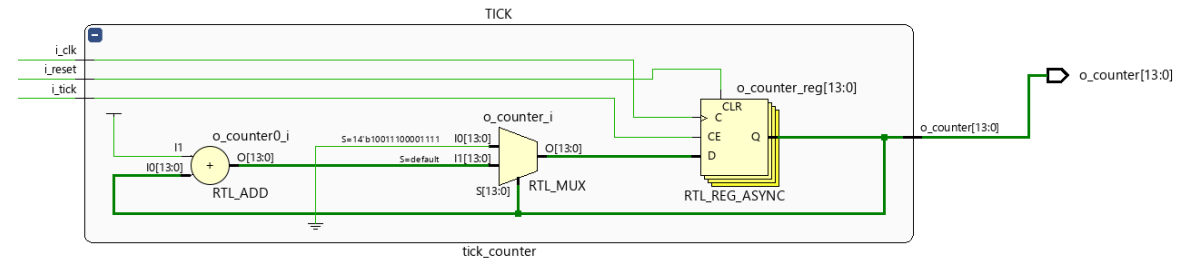
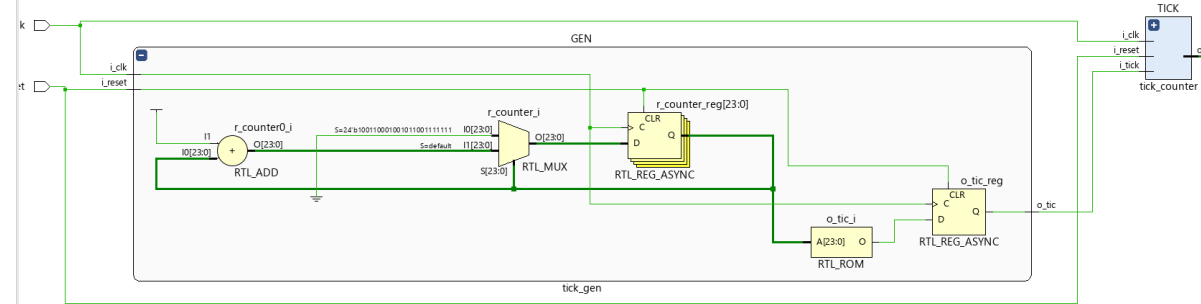
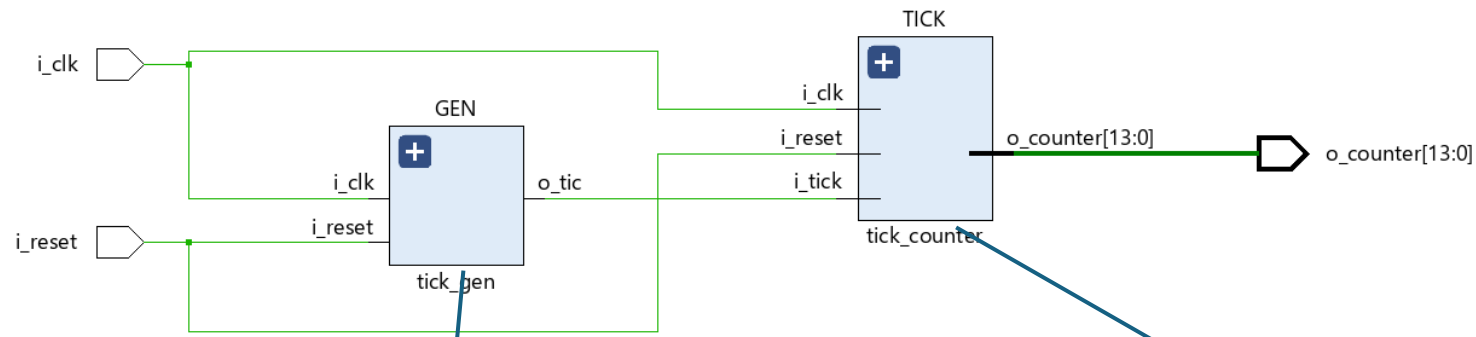
4. 조합회로와 순차논리의 특성(출력이 어떻게 다른가)

EXAMPLE

프로젝트

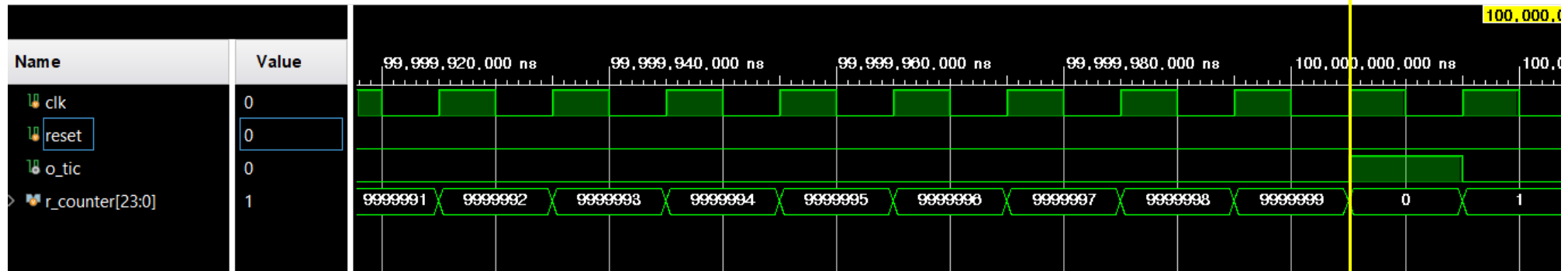
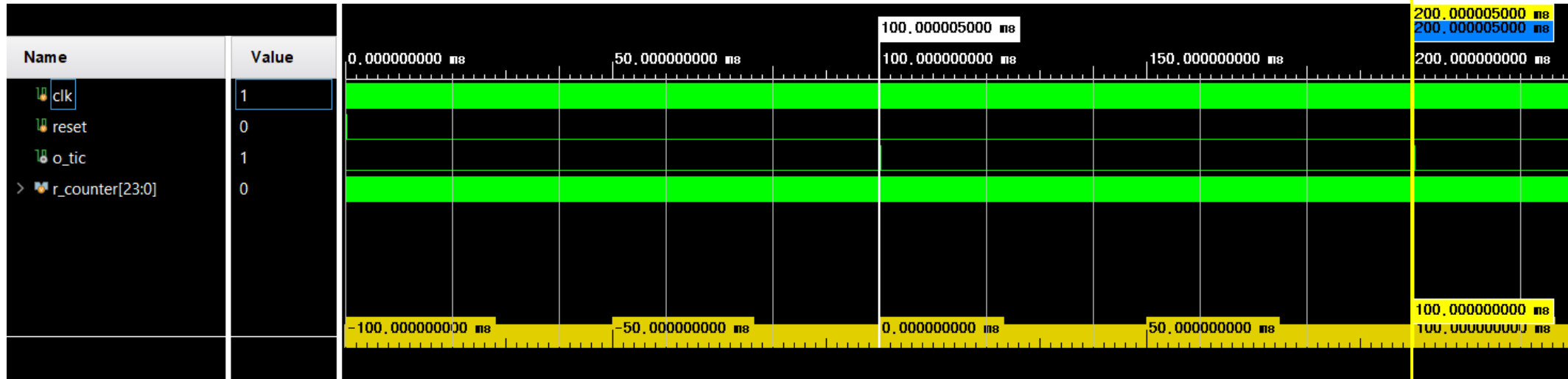
목표 : 보드 클럭 자원을 온전히 이용하여 Tick을 만들고 보드에서 9999 까지 출력 (더 추가)
조합회로/ 순차회로 구별 할것 – 색깔을 달리하면 됨. 제시된 블록다이어그램 재구성





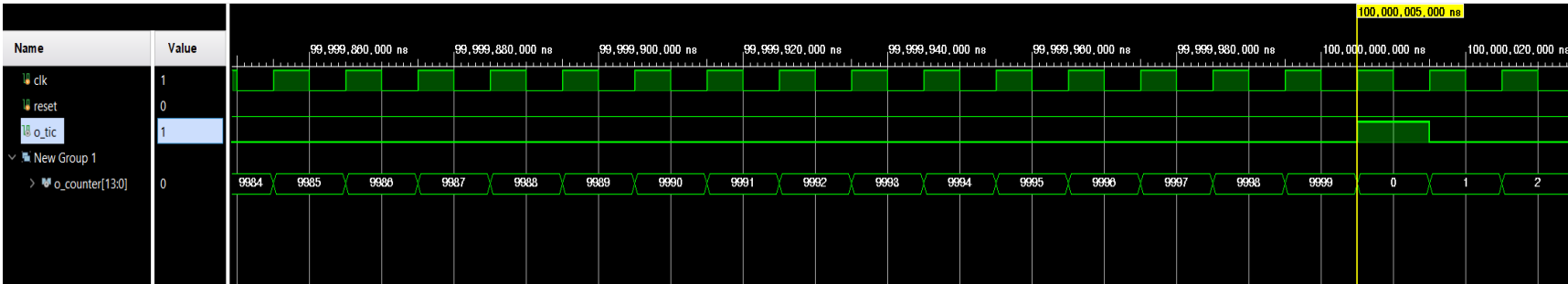
counter_datapath 내부 모습 : 모듈 별로 저 내부가 왜 저렇게 됐는지 설명할 것

4. o_tic 출력 주기 확인 결과

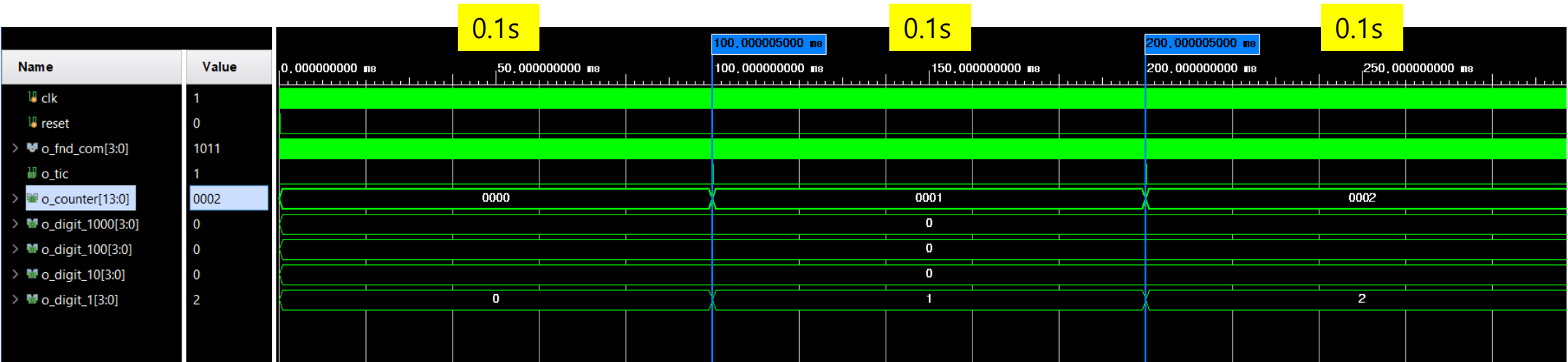


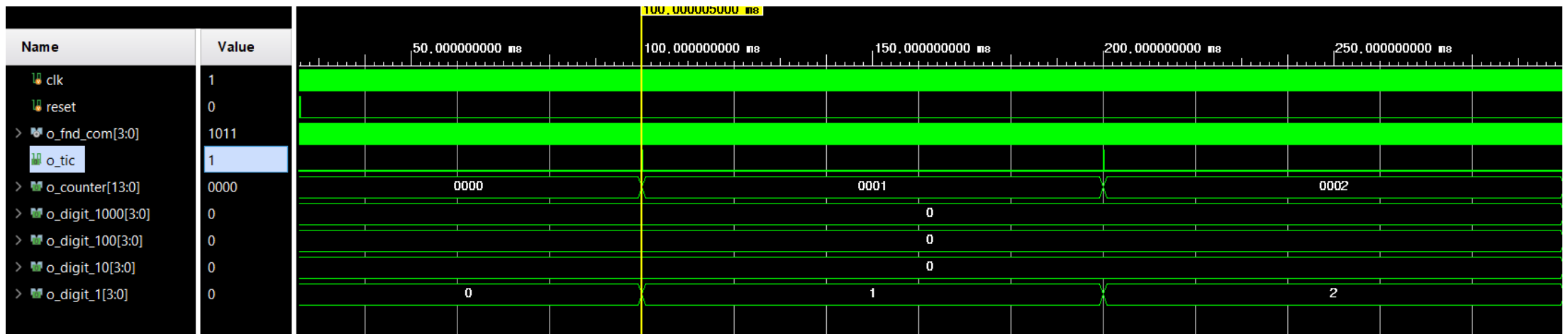
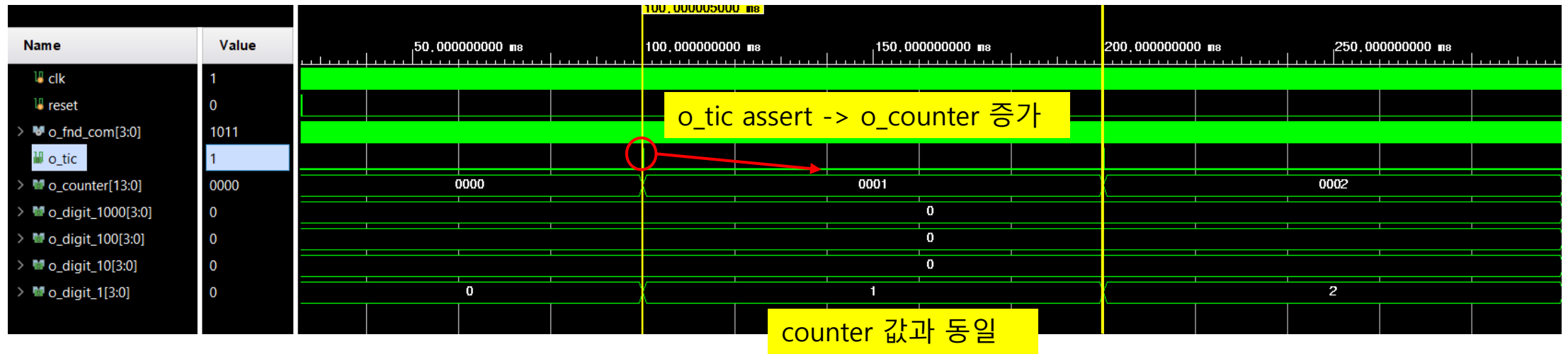
100MHz 는 1초에 100,000,000 -> 10Hz Tic 을 만드려면 tic_gen의 내부 카운터는 $100,000,000/10$ 이 필요

tic_counter 출력 결과

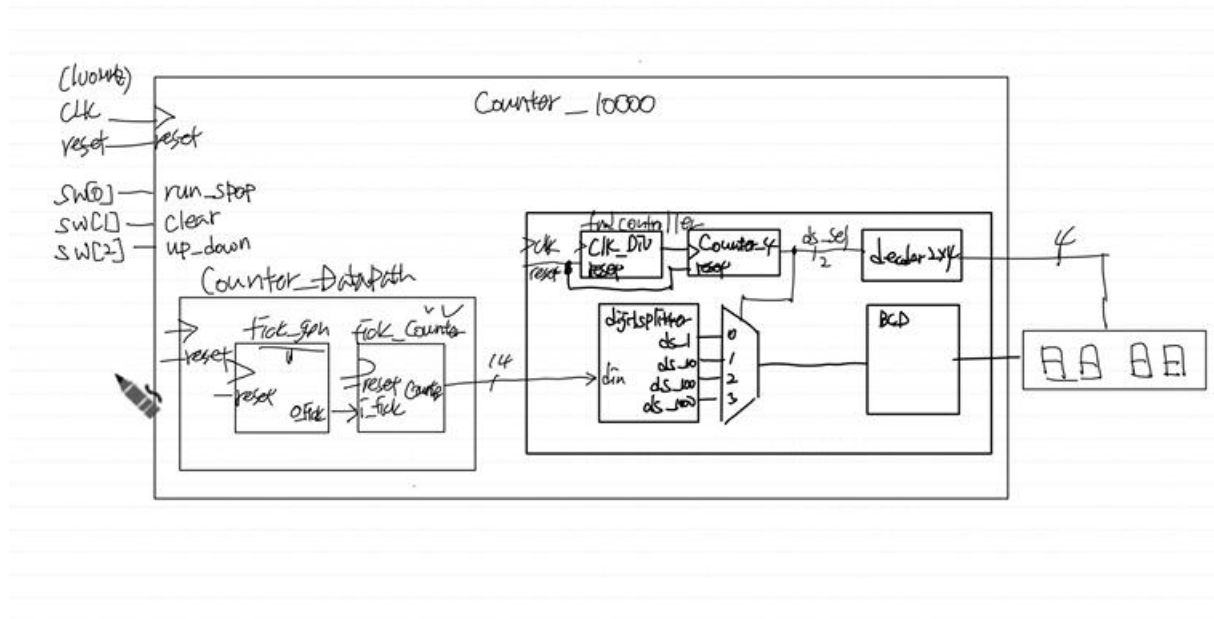


연결후 데이터 출력 결과





Counter_Datapath 모듈 재구성(3개의 입력이 들어오면 다음의 기능을 하는 모듈을 구성)



sw[0] : STOP

sw[1] : Clear

sw[2] : Counter 1씩 감소

스위치간 우선 순위는 다음과 같다

Clear

>

STOP

>

DISCOUNT

어떻게 구현했나? : i_tic이 들어오면 case 구문으로 구성

Counter_Datapath 모듈 재구성(3개의 입력이 들어오면 다음의 기능을 하는 모듈을 구성)

어떻게 구현했나? : i_tic이 들어오면 case 구문으로 구성

```
if(i_tick)begin
    case({i_sw2,i_sw1,i_sw0})
        3'b001 : begin
            o_counter <= o_counter;
        end
        3'b101 : begin
            o_counter <= o_counter;
        end
    endcase
end
```

```
////////////////////////////////////
3'b010 : begin
    o_counter <= 0;
end

3'b011 : begin
    o_counter <= 0;
end

3'b110 : begin
    o_counter <= 0;
end

3'b111 : begin
    o_counter <= 0;
end
```

```
3'b100 : begin
    if(o_counter == 0)begin o_counter <= 9999; end
    else begin o_counter <= o_counter -1 ; end
end

default : begin
    o_counter <= o_counter + 1;
    if(o_counter ==COUNT -1)begin o_counter <= 0; end
end
endcase
```

Clear

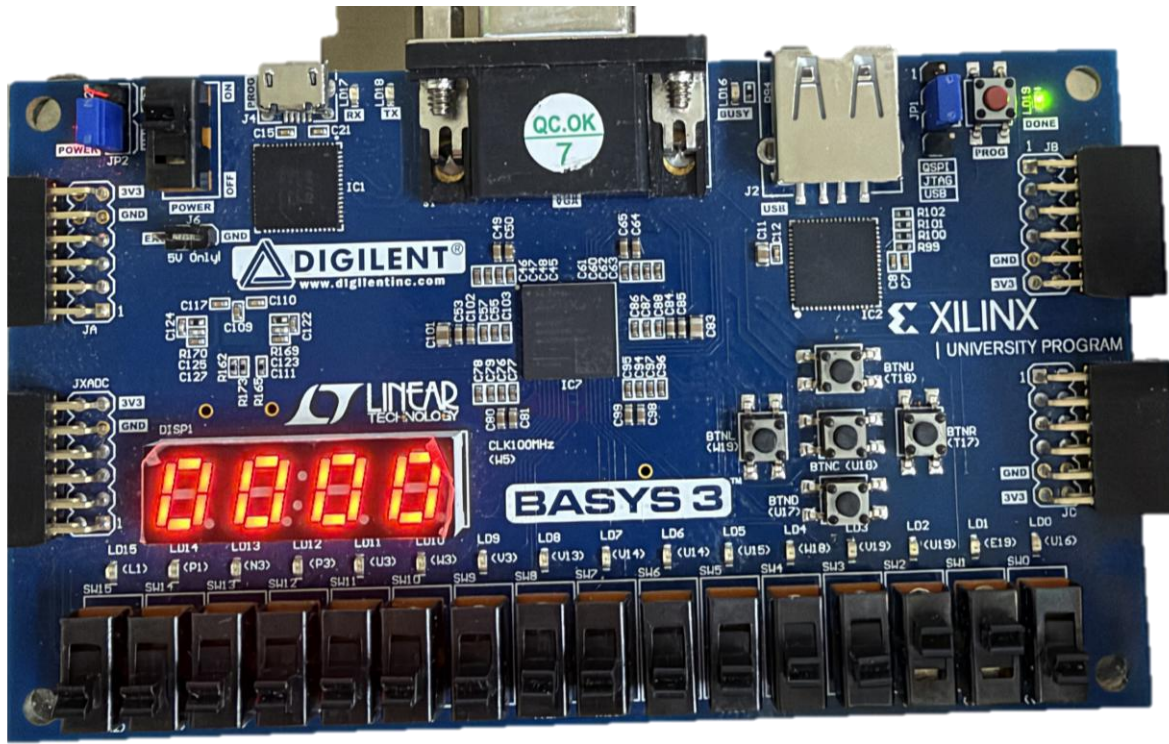
>

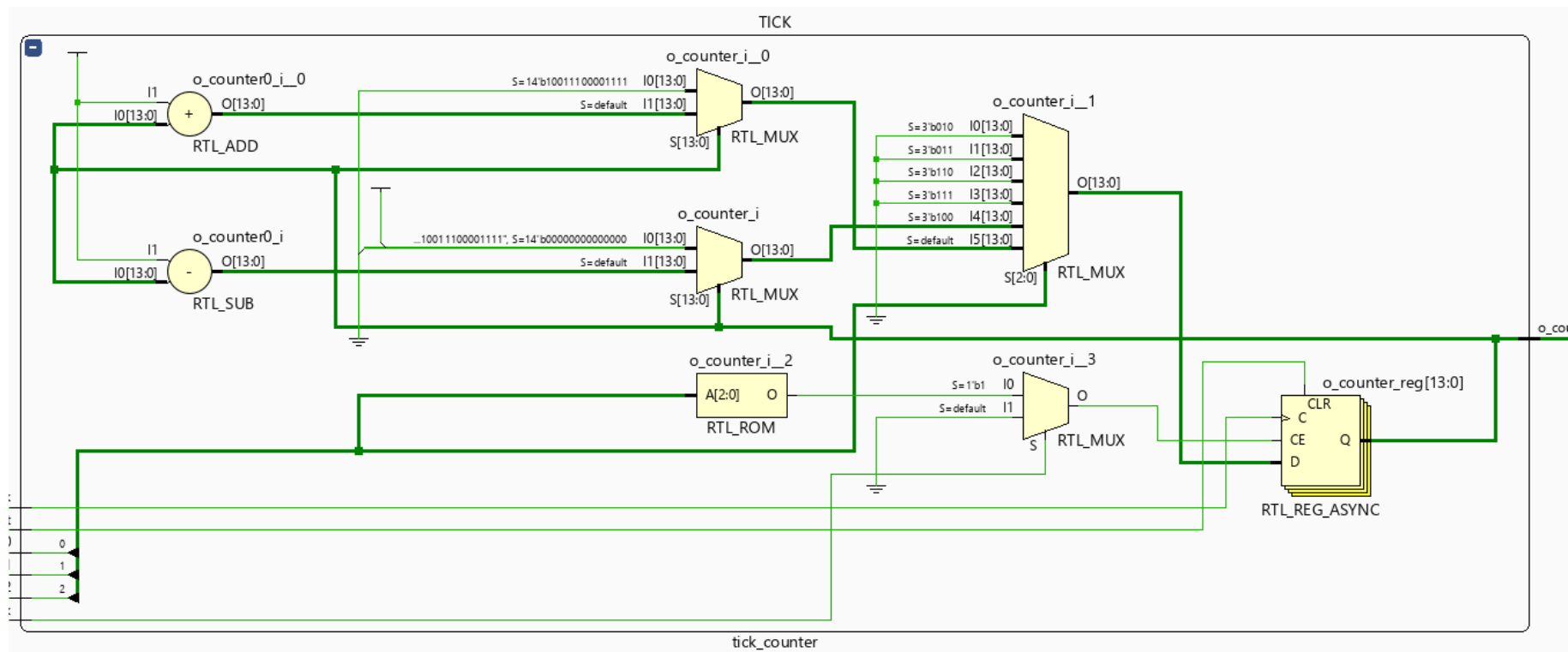
STOP

>

DISCOUNT

Board I/O 구성 – FND_COU.xdc 참조해서 표시할 것(입출력 정보)

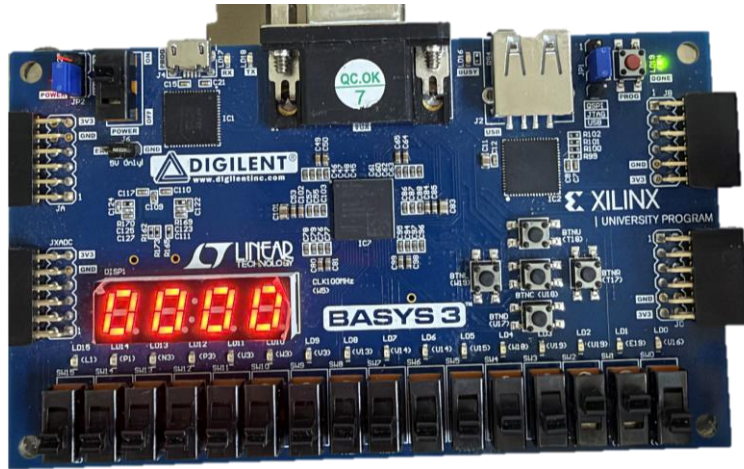




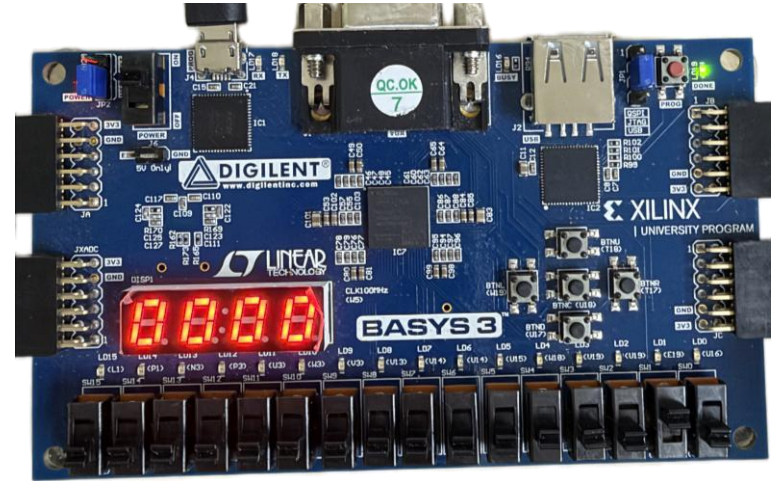
Counter_data_path_v2 내부 모습 -> 설명할 것

sim result – 경계값에서 해당기능이 잘 되나

Board results – SW[1] Clear 우선선위

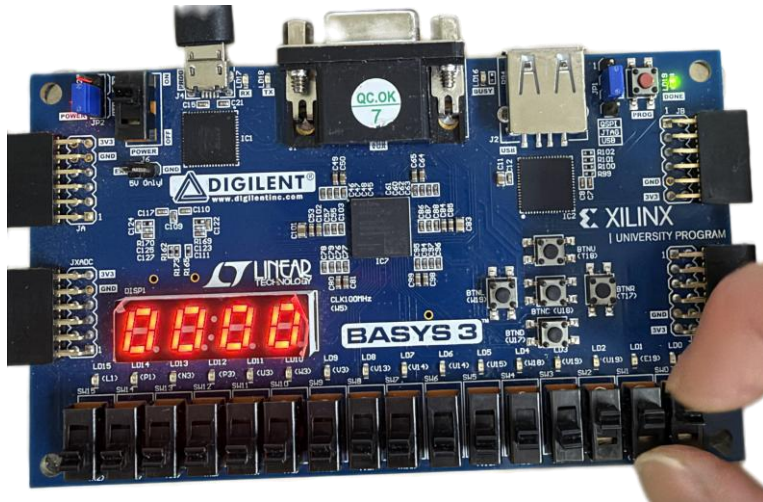


110

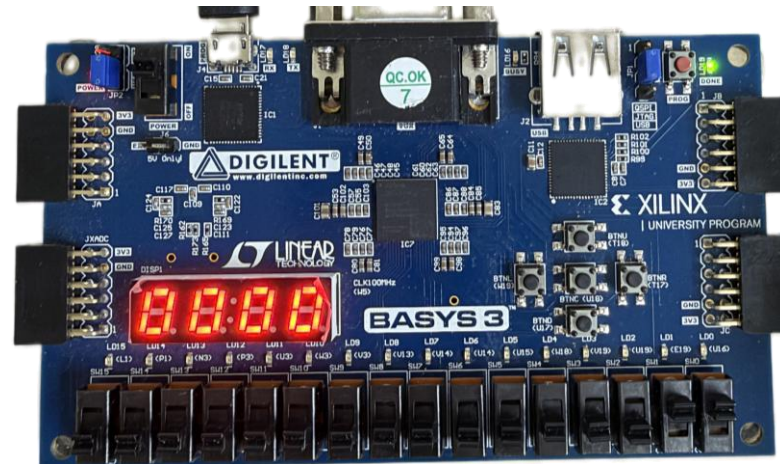


010

4가지 경우에 대해 모두 Clear 가 우선 순위

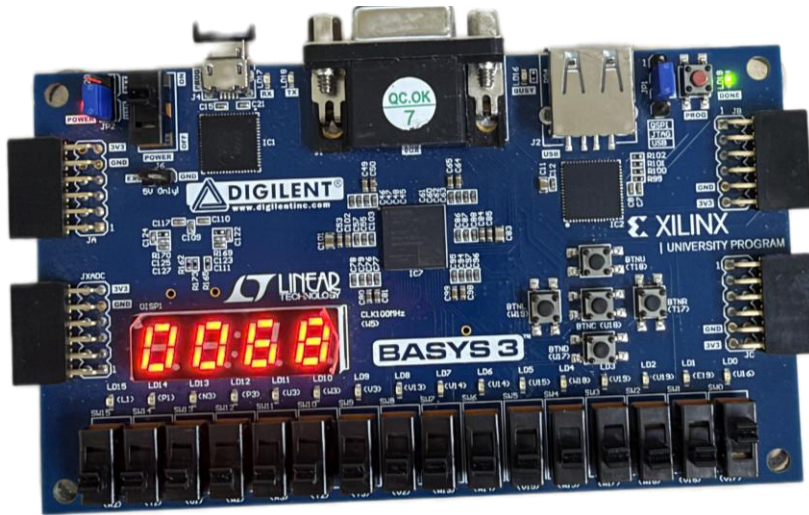


111

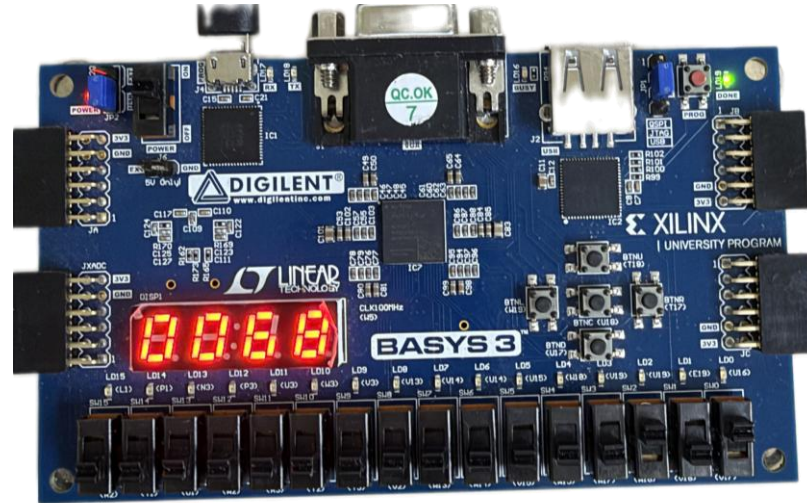


011

Board results – SW[0]



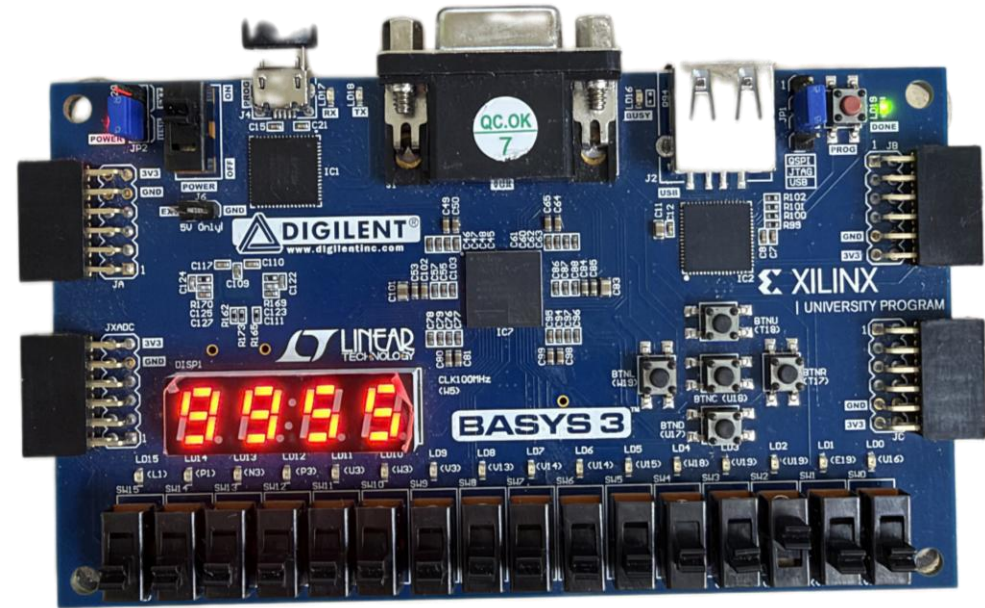
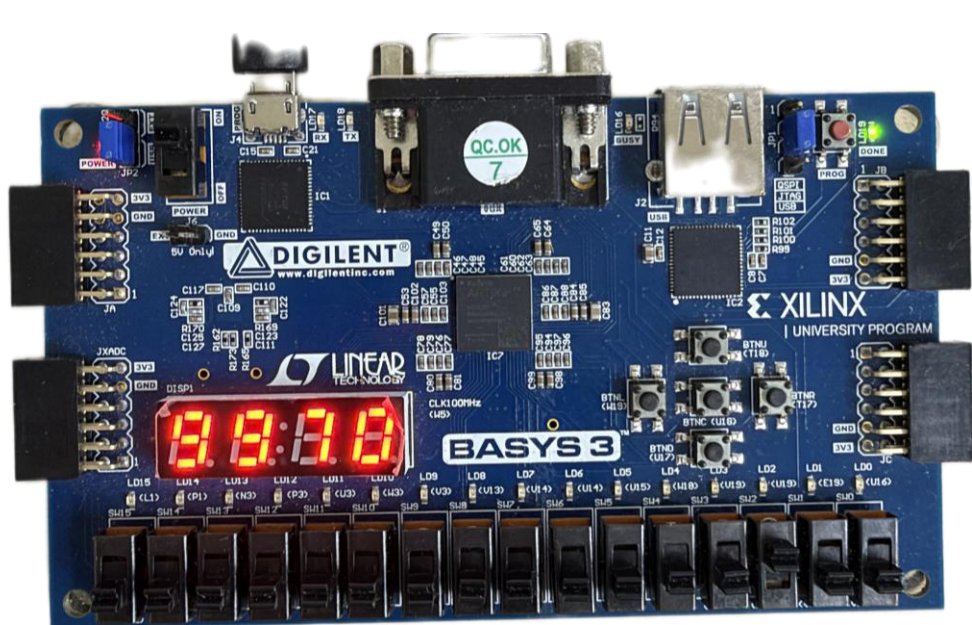
011



101

STOP과 Discount가 동시에 켜졌을 땐 STOP이 우선순위

Board results – SW[2]



Discount 작동